WEB TECHNOLOGY PROJECT REPORT

# SmartHire

AI-Powered Job Portal — Full Stack MERN Application

A production-grade job portal featuring skill-based AI recommendation engine, role-based authentication, real-time application tracking, and a modern responsive UI.

MongoDB Express.js React 18 Node.js Tailwind CSS JWT Auth REST API Vite

| SUBJECT | STACK | YEAR |
| --- | --- | --- |
| Web Technology | MERN | 2026 |

# 📋 Table of Contents


| # | Section | Description |
|---|---|---|
| 1 | Project Overview | Abstract, objectives, and scope |
| 2 | Technology Stack | Frontend, backend, database, and tools |
| 3 | System Architecture | Architecture diagram and design patterns |
| 4 | Database Schema | MongoDB collections and relationships |
| 5 | API Documentation | All REST endpoints with methods and auth |
| 6 | Recommendation Engine | AI skill-matching algorithm |
| 7 | Pages & Features | All 8 pages with functionality |
| 8 | Authentication Flow | JWT-based security implementation |
| 9 | Folder Structure | Complete project directory layout |
| 10 | Setup & Running | Installation and execution guide |
| 11 | Conclusion | Summary and future enhancements |

# 1 Project Overview

**SmartHire** is a full-stack AI-powered job portal that connects job seekers with recruiters through intelligent skill-based job matching, built on the MERN stack (MongoDB, Express, React, Node.js).

## 1.1 Abstract

SmartHire addresses the problem of irrelevant job recommendations by implementing a custom skill-matching algorithm that computes a compatibility score between a seeker's skills and a job's requirements. Unlike keyword-based search systems, this approach quantifies match quality as a percentage, enabling ranked and filtered recommendations that genuinely fit the candidate's profile.

The platform supports two user roles — **Job Seeker** and **Recruiter** — each with dedicated dashboards, profile management, and workflow tools. The application uses JWT-based authentication, bcrypt password hashing, and role-based route protection throughout.

## 1.2 Objectives

- Build a complete full-stack job portal using the MERN stack
- Implement a skill-based recommendation algorithm (match score ≥ 40%)
- Provide dual-role authentication (seeker and recruiter)
- Enable real-time application tracking with status management
- Create a responsive, modern dark-mode UI using Tailwind CSS
- Support resume upload, job bookmarking, and advanced search/filter

## 1.3 Key Features

- ✓ AI Skill-Based Job Matching (≥40% threshold)
- ✓ JWT Authentication with Role-Based Access
- ✓ Seeker & Recruiter Dashboards
- ✓ Application Tracking with Status Pipeline
- ✓ Debounced Search + Multi-Filter Jobs Page
- ✓ PDF Resume Upload (Multer)
- ✓ Job Bookmarking / Wishlist
- ✓ Paginated Job Listings (10 per page)
- ✓ Similar Jobs Recommendation on Detail Page
- ✓ Toast Notifications + Loading Spinners
- ✓ Responsive Mobile-Friendly Layout
- ✓ Recruiter Applicant Management

# 2 Technology Stack

## 2.1 Frontend

| Technology | Version | Purpose |
|---|---|---|
| React | 18.3.1 | UI component library, hooks-based state management |
| Vite | 5.3.1 | Build tool and development server |
| Tailwind CSS | 3.4.4 | Utility-first CSS framework for styling |
| React Router DOM | 6.24.0 | Client-side routing |
| Axios | 1.7.2 | HTTP client for API requests |
| React Hot Toast | 2.4.1 | Toast notification system |
| Lucide React | 0.395.0 | Icon library |

## 2.2 Backend

| Technology | Version | Purpose |
|---|---|---|
| Node.js | 18+ | JavaScript runtime environment |
| Express.js | 4.19.2 | Web application framework for REST API |
| Mongoose | 8.4.0 | MongoDB ODM for schema modeling |
| bcryptjs | 2.4.3 | Password hashing (salt rounds: 10) |
| jsonwebtoken | 9.0.2 | JWT creation and verification |
| Multer | 1.4.5-lts | Multipart file upload (resume PDFs) |
| CORS | 2.8.5 | Cross-Origin Resource Sharing |
| dotenv | 16.4.5 | Environment variable management |
| Nodemon | 3.1.3 | Auto-restart server during development |

## 2.3 Database

| Technology | Purpose |
|---|---|
| MongoDB (NoSQL) | Primary database — document storage for all collections |

| | |
|---|---|
| Mongoose ODM | Schema validation, relationships, and query building |

# 3 System Architecture

SmartHire follows a **3-tier client-server architecture** with the React SPA as the presentation layer, Express REST API as the business logic layer, and MongoDB as the data layer. Communication happens over HTTP/HTTPS using JSON.

## 3.1 Architecture Diagram

```
┌──────────────────────────────────────────────┐
│           PRESENTATION LAYER                 │
│  React 18 SPA (Vite)  │  Port: 5173          │
│  ┌──────────┐ ┌──────────┐ ┌────────────┐    │
│  │  Pages   │ │Components│ │  Context   │    │
│  │  (8 pgs) │ │(Navbar,  │ │(AuthContext│    │
│  │          │ │ JobCard) │ │ JWT state) │    │
│  └──────────┘ └──────────┘ └────────────┘    │
│         Axios HTTP Client + JWT Header       │
└──────────────────┬───────────────────────────┘
                   │  HTTP REST API  (CORS)
┌──────────────────▼───────────────────────────┐
│            BUSINESS LOGIC LAYER              │
│  Express.js REST API  │  Port: 5000          │
│  ┌────────┐ ┌──────────────┐ ┌────────────┐  │
│  │ Routes │ │ Controllers  │ │ Middleware │  │
│  │/auth   │ │authController│ │JWT protect │  │
│  │/jobs   │ │jobController │ │Role guard  │  │
│  │/apps   │ │appController │ │Multer      │  │
│  │/profile│ │profileCtrl   │ │Error hdlr  │  │
│  └────────┘ └──────────────┘ └────────────┘  │
│  ┌──────────────────────────────────────┐    │
│  │  utils/matchScore.js (AI Engine)     │    │
│  └──────────────────────────────────────┘    │
└──────────────────┬───────────────────────────┘
                   │  Mongoose ODM
┌──────────────────▼───────────────────────────┐
│               DATA LAYER                     │
│  MongoDB  │  Port: 27017  │  DB: smarthire   │
│  ┌──────┐ ┌────┐ ┌──────┐ ┌────────────┐    │
│  │Users │ │Jobs│ │Apps  │ │ Bookmarks  │    │
│  └──────┘ └────┘ └──────┘ └────────────┘    │
│  ┌──────────────┐ ┌────────────────────┐     │
│  │ SeekerProfiles│ │  RecruiterProfiles  │    │
│  └──────────────┘ └────────────────────┘     │
└──────────────────────────────────────────────┘
```

### 3.2 Request-Response Flow

User Action → React Component → Axios + JWT → Express Route →

Auth Middleware → Controller → MongoDB → JSON Response

# 4 Database Schema

## 4.1 User Collection

| Field | Type | Constraints | Description |
|---|---|---|---|
| _id | ObjectId | Auto-generated | Primary key |
| name | String | Required, min 2 chars | Full name |
| email | String | Required, unique, lowercase | Login email |
| password | String | Required, hashed | bcrypt hash (10 rounds) |
| role | String | Enum: seeker\|recruiter\|admin | User role |
| createdAt | Date | Auto timestamp | Registration date |

## 4.2 SeekerProfile Collection

| Field | Type | Description |
|---|---|---|
| userId | ObjectId (ref: User) | One-to-one relationship with User |
| skills | [String] | Array of skill tags e.g. ["React","Node.js"] |
| experience | String | Years/description of experience |
| location | String | Current city/country |
| bio | String | Personal summary |
| resumeUrl | String | Server path to uploaded PDF |
| resumeOriginalName | String | Original file name |

## 4.3 Job Collection

| Field | Type | Description |
|---|---|---|
| title | String | Job title (indexed for text search) |
| company | String | Company name |
| location | String | Job location |
| salaryMin | Number | Minimum annual salary in USD |

| | | |
|---|---|---|
| salaryMax | Number | Maximum annual salary in USD |
| type | String | Full-time \| Part-time \| Remote \| Contract \| Internship |
| requiredSkills | [String] | Skills used for matching algorithm |
| description | String | Full job description |
| recruiterId | ObjectId (ref: User) | Posted by recruiter |
| isActive | Boolean | Soft delete / visibility flag |

### 4.4 Application Collection

| Field | Type | Description |
|---|---|---|
| jobId | ObjectId (ref: Job) | Applied job reference |
| seekerId | ObjectId (ref: User) | Applicant reference |
| status | String | Applied \| Under Review \| Shortlisted \| Rejected |
| appliedAt | Date | Timestamp of application |

*Compound unique index on {jobId, seekerId} prevents duplicate applications.*

### 4.5 Bookmark Collection

| Field | Type | Description |
|---|---|---|
| userId | ObjectId (ref: User) | User who bookmarked |
| jobId | ObjectId (ref: Job) | Bookmarked job |

*Compound unique index on {userId, jobId} prevents duplicate bookmarks.*

# 5 API Documentation

## 5.1 Authentication Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| POST | /api/auth/register | Public | Register user + auto-create profile |
| POST | /api/auth/login | Public | Login → returns JWT token |
| GET | /api/auth/me | JWT | Get current authenticated user |

## 5.2 Job Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| GET | /api/jobs | Public | List jobs (search, filter, paginate) |
| GET | /api/jobs/:id | Public | Job detail + similar jobs |
| GET | /api/jobs/recommended | Seeker | AI-matched jobs for seeker |
| GET | /api/jobs/my-jobs | Recruiter | Recruiter's own job postings |
| GET | /api/jobs/:id/applicants | Recruiter | All applicants for a job |
| POST | /api/jobs | Recruiter | Create new job posting |
| PUT | /api/jobs/:id | Recruiter | Update job (owner only) |
| DELETE | /api/jobs/:id | Recruiter | Delete job (owner only) |

## 5.3 Application Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| POST | /api/applications | Seeker | Apply to a job |
| GET | /api/applications | Seeker | Get my applications |
| GET | /api/applications/check/:jobId | Seeker | Check if already applied |
| PUT | /api/applications/:id/status | Recruiter | Update application status |

## 5.4 Profile Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| GET | /api/profile/seeker | Seeker | Get seeker profile |
| PUT | /api/profile/seeker | Seeker | Update + upload resume (multipart) |
| GET | /api/profile/recruiter | Recruiter | Get recruiter profile |
| PUT | /api/profile/recruiter | Recruiter | Update recruiter profile |

## 5.5 Bookmark Endpoints

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| GET | /api/bookmarks | JWT | Get all bookmarked jobs |
| POST | /api/bookmarks/:jobId | JWT | Bookmark a job |
| DELETE | /api/bookmarks/:jobId | JWT | Remove bookmark |

## 5.6 Sample API Request / Response

### POST /api/auth/login

```
// Request Body
{
  "email": "john@example.com",
  "password": "mypassword123"
}

// Response (200 OK)
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "665f1a2b3c4d5e6f7g8h9i0j",
    "name": "John Doe",
    "email": "john@example.com",
    "role": "seeker"
  }
}
```

# 6 AI Recommendation Engine

📐 **Match Score Formula**

**Match Score = (Number of Matching Skills ÷ Total Required Skills) × 100**

A job is recommended if its match score is ≥ 40%. Results are sorted in descending order by score.

## 6.1 Algorithm Implementation

```
// server/utils/matchScore.js

function matchScore(seekerSkills, jobSkills) {
  if (!jobSkills || jobSkills.length === 0) return 0;
  if (!seekerSkills || seekerSkills.length === 0) return 0;

  // Case-insensitive comparison
  const normalizedSeeker = seekerSkills.map(s => s.toLowerCase().trim());
  const matches = jobSkills.filter(skill =>
    normalizedSeeker.includes(skill.toLowerCase().trim())
  );

  return Math.round((matches.length / jobSkills.length) * 100);
}

function getRecommendedJobs(seekerSkills, jobs, threshold = 40) {
  return jobs
    .map(job => ({
      ...job.toObject(),
      matchScore: matchScore(seekerSkills, job.requiredSkills)
    }))
    .filter(job => job.matchScore >= threshold)  // 40% minimum
    .sort((a, b) => b.matchScore - a.matchScore); // descending
}
```

## 6.2 Example Calculation

| Seeker Skills | Job Required Skills | Matches | Score | Recommended? |
|---|---|---|---|---|
| React, Node.js, MongoDB, CSS | React, Node.js, MongoDB, Express | 3/4 | **75%** | ✅ Yes |
| React, CSS, HTML | React, Node.js, MongoDB, Express | 1/4 | **25%** | ❌ No |
| Python, Django, PostgreSQL | Python, Django, REST API | 2/3 | **67%** | ✅ Yes |

| | | | | |
|---|---|---|---|---|
| Java, Spring | React, Node.js, AWS, Docker | 0/4 | **0%** | ❌ No |

## 6.3 Match Score Badge Colors

| Score Range | Badge Color | Meaning |
|---|---|---|
| 80% – 100% | 🟢 Green | Excellent match |
| 60% – 79% | 🔵 Teal | Good match |
| 40% – 59% | 🔷 Blue | Moderate match |
| Below 40% | — | Not shown (filtered out) |

# 7 Pages & Features

### 🏠 Landing Page (/)

- Animated hero with gradient text
- 4 live stats (Jobs, Companies, etc.)
- Latest 6 featured job cards
- "How It Works" 3-step section
- Testimonials from 3 users
- CTA banner with sign-up links

### 🔑 Auth Pages (/login, /register)

- Role toggle: Job Seeker / Recruiter
- Client-side validation on all fields
- Password show/hide toggle
- Loading spinner on submit
- Role-based redirect after login
- JWT stored in localStorage

### 👤 Seeker Dashboard

- Stats: applications, shortlisted, matches, skills
- Profile tab: edit location, experience, bio
- Skill tag input (add/remove)
- PDF resume upload with drag-style UI
- AI Matches tab with % badges
- Applications tab with status table

### 🏢 Recruiter Dashboard

- Post job form with skill tag input
- My Jobs list with edit/delete
- View applicants per job
- Inline status selector per applicant
- Stats: active jobs, account type
- Tabbed navigation (Jobs / Applicants)

### 🔍 Jobs Listing (/jobs)

- Debounced search (300ms delay)
- Expandable filter panel
- Filter: location, type, min salary
- Active filter chips with remove
- Bookmark toggle per card
- Pagination (10 per page)

### 📄 Job Detail (/jobs/:id)

- Full job description
- Skill tags display
- One-click apply button
- Save/bookmark button
- Sticky sidebar with salary info
- Similar jobs grid (4 cards)

### 📊 Applications (/applications)

- 4 stat cards: total, review, shortlisted, rejected
- Pipeline progress bars per status
- Responsive table with all applications
- Color-coded StatusBadge per row
- Link to job detail from table
- Refresh button for live updates

### 🧭 Navbar

- Sticky with glassmorphism blur
- Gradient SmartHire logo
- Avatar dropdown when logged in
- Role-based nav links
- Mobile hamburger menu
- Active link highlighting

# 8 Authentication Flow

## 8.1 Registration Flow

Fill Form → Validate Client → POST /register → Validate Server → Hash Password → Create User → Create Profile → Return JWT

## 8.2 JWT Token Lifecycle

```
// Token generation (server)
const token = jwt.sign(
  { id: user._id },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }   // 7-day expiry
);

// Token storage (client)
localStorage.setItem('smarthire_token', token);

// Auto-attach on every request (Axios interceptor)
api.interceptors.request.use(config => {
  const token = localStorage.getItem('smarthire_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auto-logout on 401 (Axios interceptor)
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.clear();
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

## 8.3 Route Protection Layers

| Layer | Location | How it works |
|---|---|---|
| Frontend Route Guard | PrivateRoute.jsx | Checks AuthContext — redirects to /login if no user |

| | | |
|---|---|---|
| Role Guard | PrivateRoute (role prop) | Redirects to / if user role doesn't match required role |
| Backend JWT Guard | middleware/auth.js → protect() | Verifies Bearer token, attaches req.user |
| Backend Role Guard | middleware/auth.js → authorize() | Checks req.user.role against allowed roles |

# 9 Project Folder Structure

```
WT 9346/
├── README.md
│
├── server/                        ← Node.js + Express Backend
│   ├── server.js                  ← Entry point, MongoDB connect, route mount
│   ├── package.json
│   ├── .env                       ← PORT, MONGO_URI, JWT_SECRET
│   ├── uploads/                   ← Uploaded resume PDFs (auto-created)
│   │
│   ├── models/
│   │   ├── User.js                ← name, email, password, role
│   │   ├── SeekerProfile.js       ← skills[], experience, bio, resumeUrl
│   │   ├── RecruiterProfile.js    ← companyName, website
│   │   ├── Job.js                 ← title, company, requiredSkills[], etc.
│   │   ├── Application.js         ← jobId, seekerId, status
│   │   └── Bookmark.js            ← userId, jobId
│   │
│   ├── controllers/
│   │   ├── authController.js      ← register(), login(), getMe()
│   │   ├── jobController.js       ← CRUD + search + recommend + applicants
│   │   ├── applicationController.js← apply, list, updateStatus, check
│   │   ├── profileController.js   ← get/update seeker+recruiter + multer
│   │   └── bookmarkController.js  ← add, remove, list
│   │
│   ├── routes/
│   │   ├── auth.js
│   │   ├── jobs.js
│   │   ├── applications.js
│   │   ├── profile.js
│   │   └── bookmarks.js
│   │
│   ├── middleware/
│   │   └── auth.js                ← protect() + authorize() guards
│   │
│   └── utils/
│       └── matchScore.js          ← matchScore() + getRecommendedJobs()
│
└── client/                        ← React 18 + Vite + Tailwind Frontend
    ├── index.html                 ← Root HTML with Inter font
    ├── vite.config.js             ← Proxy: /api → localhost:5000
    ├── tailwind.config.js         ← Custom colors + animations
    ├── postcss.config.js
    ├── package.json
    │
    └── src/
        ├── main.jsx               ← ReactDOM.createRoot entry
        ├── App.jsx                ← BrowserRouter + all Routes + Toaster
```

```
├── index.css              ← Tailwind + custom component classes
│
├── context/
│   └── AuthContext.jsx    ← user state, login(), logout()
│
├── hooks/
│   └── useDebounce.js     ← 300ms debounce utility
│
├── utils/
│   └── api.js             ← Axios instance + JWT interceptors
│
├── components/
│   ├── Navbar.jsx         ← Sticky, glassmorphism, dropdown
│   ├── JobCard.jsx        ← Card with match badge, bookmark
│   ├── PrivateRoute.jsx   ← Auth + role route guard
│   ├── StatusBadge.jsx    ← Color-coded status pill
│   ├── MatchBadge.jsx     ← AI match percentage badge
│   ├── Pagination.jsx     ← Page navigation with ellipsis
│   ├── EmptyState.jsx     ← No-data illustration placeholder
│   └── Spinner.jsx        ← Loading spinner (4 sizes)
│
└── pages/
    ├── Landing.jsx        ← Hero, features, testimonials
    ├── Login.jsx          ← Email/password form
    ├── Register.jsx       ← Role toggle + registration form
    ├── SeekerDashboard.jsx ← Profile, matches, applications tabs
    ├── RecruiterDashboard.jsx← Post jobs, applicant management
    ├── Jobs.jsx           ← Search, filter, paginate jobs
    ├── JobDetail.jsx      ← Full job + apply + similar jobs
    └── Applications.jsx   ← Pipeline + status table
```

# 10 Setup & Running Guide

## 10.1 Prerequisites

| Requirement | Minimum Version | Check Command |
|---|---|---|
| Node.js | v18.0+ | `node -v` |
| npm | v9.0+ | `npm -v` |
| MongoDB | v6.0+ | `mongod --version` |

## 10.2 Environment Configuration

Create/edit `server/.env` :

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/smarthire
JWT_SECRET=smarthire_super_secret_jwt_key_2024
NODE_ENV=development
```

## 10.3 Running the Application

**Terminal 1 — Backend:**

```
cd "C:\Users\aryan\OneDrive\Desktop\WT 9346\server"
npm install          # first time only
npm run dev
```

**Terminal 2 — Frontend:**

```
cd "C:\Users\aryan\OneDrive\Desktop\WT 9346\client"
npm install          # first time only
npm run dev
```

Open browser: **http://localhost:5173**

## 10.4 Test Accounts (Create via Register page)

| Role | Register at | Dashboard |
|---|---|---|
| Job Seeker | /register → Select "Job Seeker" | /dashboard/seeker |
| Recruiter | /register → Select "Recruiter" | /dashboard/recruiter |

# 11 Conclusion

SmartHire successfully demonstrates a production-grade, full-stack MERN application with intelligent job matching, dual-role authentication, and a polished user experience.

## 11.1 What Was Achieved

- Built a complete 3-tier MERN application from scratch with clean separation of concerns
- Implemented a custom skill-based recommendation engine with configurable match threshold
- Created 6 MongoDB collections with proper indexing and relationships
- Delivered 16 REST API endpoints with JWT auth and role-based access control
- Built 8 fully functional React pages with responsive design and micro-animations
- Used modern tooling: Vite for fast HMR, Tailwind for rapid UI development, Axios interceptors for auth

## 11.2 Challenges & Solutions

| Challenge | Solution |
|---|---|
| Preventing duplicate applications | Compound unique index on {jobId, seekerId} in Application schema |
| Route ordering conflict (recommended vs :id) | Placed named routes before parameterized routes in Express |
| Case-insensitive skill matching | Normalized both arrays to lowercase before comparison |
| JWT persisting across page refreshes | Stored token in localStorage and verified on AuthContext mount |
| Multer file validation | MIME type check (application/pdf) + 5MB size limit |

## 11.3 Future Enhancements

- **Admin Panel** — View all users, jobs, and applications with moderation controls
- **Real-time Notifications** — Socket.io for instant status updates
- **Advanced AI Matching** — NLP-based semantic skill similarity (not just exact match)
- **Email Notifications** — Nodemailer for application status emails
- **Cloud Deployment** — Deploy to Railway/Render (backend) + Vercel (frontend) + MongoDB Atlas
- **Interview Scheduling** — Calendar integration for shortlisted candidates

## 11.4 Project Metrics

### Backend

- 6 MongoDB models

### Frontend

- 8 page components

- 5 route files
- 5 controller files
- 16 REST API endpoints
- 1 recommendation utility
- 1 auth middleware

- 8 shared components
- 1 context (AuthContext)
- 1 custom hook (useDebounce)
- 1 Axios utility (api.js)
- Fully responsive on all devices

---

**SmartHire — AI-Powered Job Portal**

Built with MongoDB • Express.js • React 18 • Node.js • Tailwind CSS

Web Technology Project Report | 2026